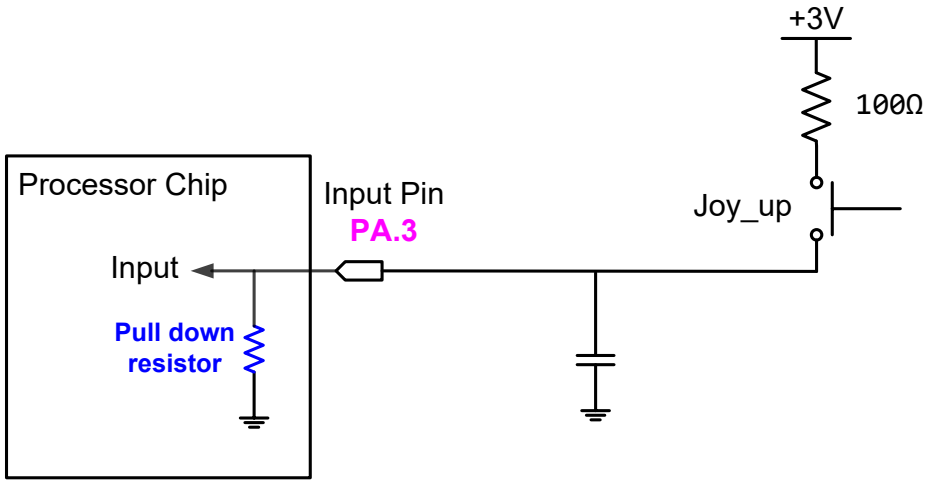
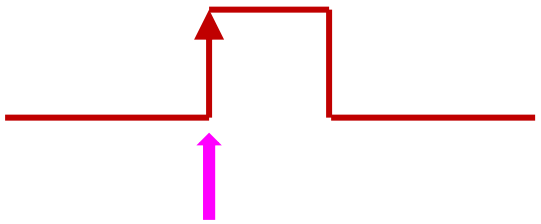
External Interrupt(EXTI) Register Description

EXTI Register

- **External Interrupt**
  - For interrupts triggered by peripherals: e.g. switches, buttons
  - EXTI0 to EXTI15 : total 16
  - Each EXTIx is matched to Pinx of a GPIO port
    - E.g. EXTI0 is matched to only one pin from PA0, PB0.. or PG0

  - Detects trigger(rising/falling edge) → trigger EXTIx →
    → Request to NVIC→ EXTIx_IRQHandler()

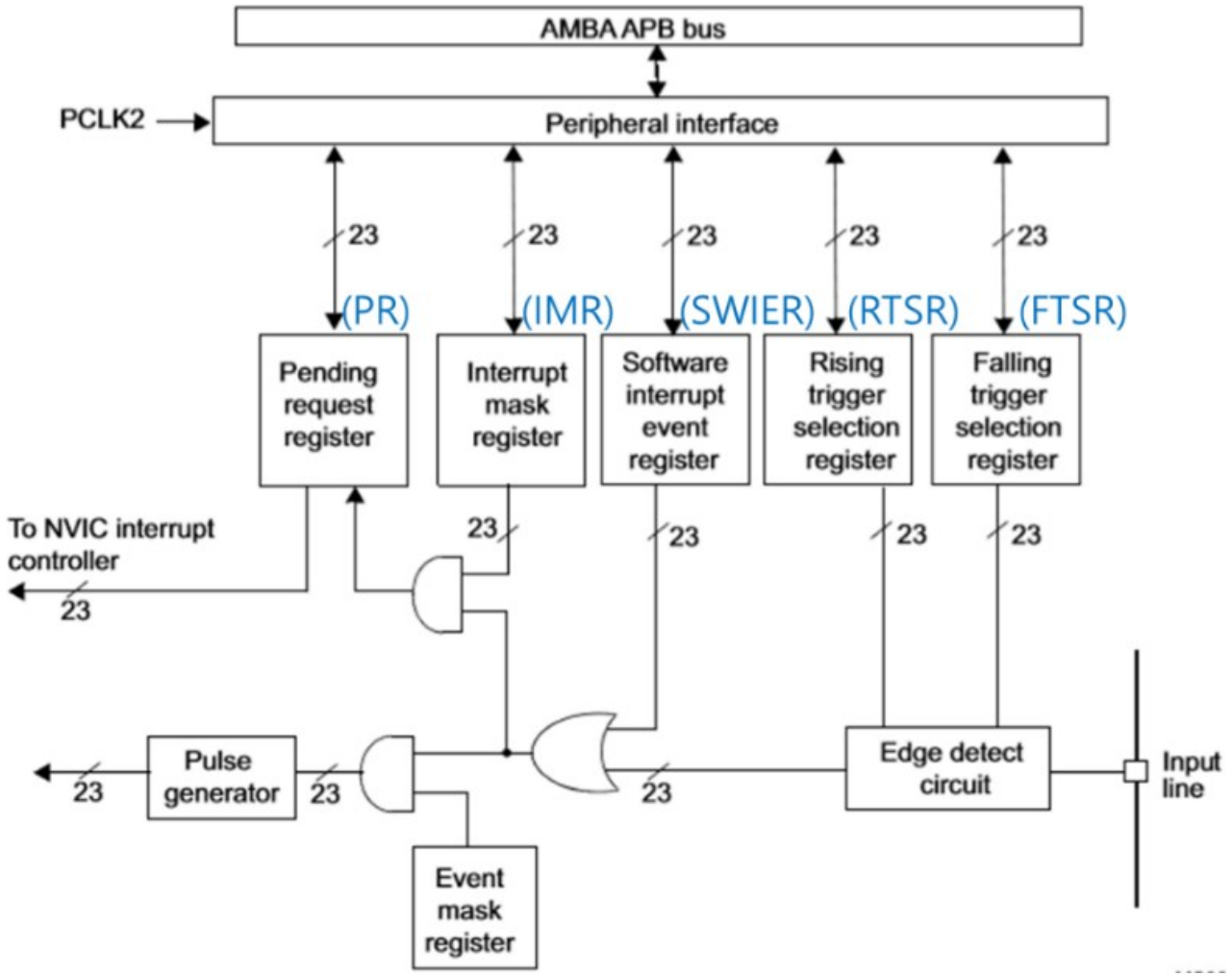




Rising edge triggers EXTI interrupt.

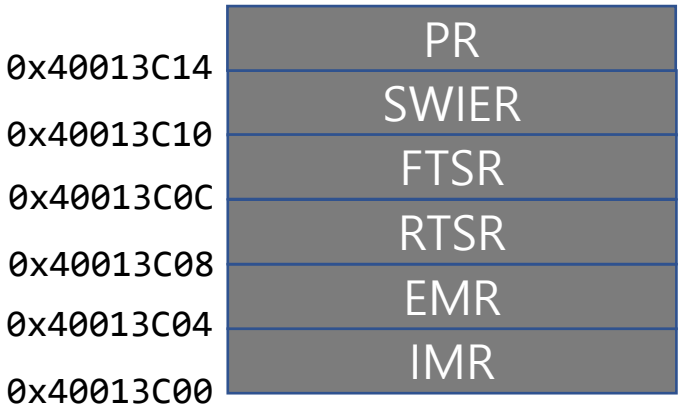
EXTI Register

- **EXTI Register List**
  - **Control register**
    SYSCFG_EXTICR
  - **Mask register**
    EXTI_IMR
    EXTI_EMR
  - **Trigger registers**
    EXTI_FTSR
    EXTI_RTSR
  - **Pending register**
    EXTI_PR



EXTI Register

- **EXTI Register Memory Address**




```
typedef struct
{
  __IO uint32_t IMR;    // EXTI Interrupt mask register
  __IO uint32_t EMR;    // EXTI Event mask register
  __IO uint32_t RTSR;   // EXTI Rising trigger selection register
  __IO uint32_t FTSR;   // EXTI Falling trigger selection register
  __IO uint32_t SWIER;  // EXTI Software interrupt event register
  __IO uint32_t PR;     // EXTI Pending register
}

#define EXTI ((EXTI_TypeDef *) 0x40013C00)
```

EXTI Register

- **EXTI Configuration**

(Digital Input Setting)

- Enable GPIO peripheral clock (**RCC->AHB1ENR**)
- Configure digital input pin(push-button) using GPIO registers.

e.g. PC13(user button) 설정

(EXTI Setting)

- Enable SYSCFG peripheral clock. (**RCC->APB2ENR**)
- Connect the corresponding external line to GPIO (**SYSCFG->EXTICR**)
- Configure the trigger edge. (**EXTI->FTSR/RTSR**)
- Configure Interrupt mask (**EXTI->IMR**)
- Enable EXTI. (**EXTI->IMR**)

GPIO Clock Enable과 다름

SYSCFG_TypeDef 에서 정의됨

(NVIC Setting)

- Configure the priority of EXTI interrupt request. (**NVIC_SetPriority**)
- Enable EXTI interrupt request. (**NVIC_EnableIRQ**)

(EXTI Use)

- Create user codes in handler **EXTI_IRQHandler()**
- Clear pending bit after interrupt call

EXTI Register

- **EXTI Setting:**
  - Enable (SYSCFG) System configuration controller clock (for EXTI)

    RCC->APB2ENR: SYSCFGEN

Example:
PC_13(user button) EXTI

```
RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
```

6.3.12 RCC APB2 peripheral clock enable register (RCC_APB2ENR)

Address offset: 0x44

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved											SPI5EN	Reser-ved	TIM11 EN	TIM10 EN	TIM9 EN
											rw		rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reser-ved	SYSCFG EN	SPI4EN	SPI1 EN	SDIO EN	Reserved		ADC1 EN	Reserved		USART6 EN	USART1 EN	Reserved			TIM1 EN
	rw	rw	rw	rw			rw			rw	rw				rw

Bit 14 **SYSCFGEN:** System configuration controller clock enable

Set and cleared by software.

0: System configuration controller clock disabled

1: System configuration controller clock enabled

EXTI Register

- **EXTI Setting:**
  - EXTIx Connection GPIO Pinx: (**SYSCFG_EXTICR**)

    Connect EXTI13 to PC_13 in EXTICR[3]

7.2.6 SYSCFG external interrupt configuration register 4 (SYSCFG_EXTICR4)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI15[3:0]				EXTI14[3:0]				EXTI13[3:0]				EXTI12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 12 to 15)

These bits are written by software to select the source input for the EXTIx external interrupt.

0000: PA[x] pin
0001: PB[x] pin
0010: PC[x] pin
0011: PD[x] pin
0100: PE[x] pin
0101: Reserved
0110: Reserved
0111: PH[x] pin

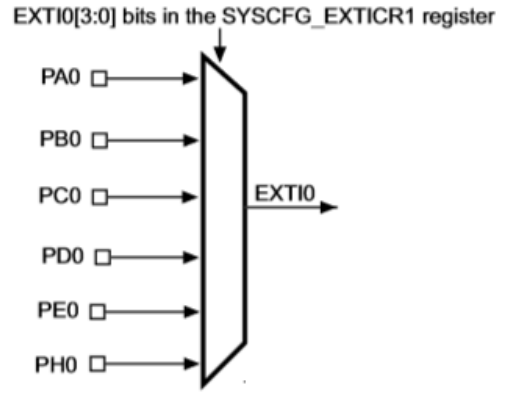
Example:

PC_13(user button) EXTI, PC[x] pin, x=13

SYSCFG_EXTICR4 is SYSCFG –>EXTICR[3]

```
SYSCFG->EXTICR[3] &= ~SYSCFG_EXTICR4_EXTI13;
SYSCFG->EXTICR[3] |=  SYSCFG_EXTICR4_EXTI13_PC;
```

Note: EXTI0 for Px_0 is in EXTICR[0]



EXTI Register

- **EXTI Setting**

- Configure the trigger edge: (**EXTI_FTSR/RTSR**)

Example:
PC_13(user button) EXTI
Falling Edge Trigger

```
EXTI->FTSR |= EXTI_FTSR_TR13;
```

10.3.4 Falling trigger selection register (EXTI_FTSR)

Address offset: 0x0C
Reset value: 0x0000 0000

User button: pull-up

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									TR22	TR21	Reserved		TR18	TR17	TR16
									rw	rw			rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:0 **TRx:** Falling trigger event configuration bit of line x

0: Falling trigger disabled (for Event and Interrupt) for input line

1: Falling trigger enabled (for Event and Interrupt) for input line.

Note: The external wakeup lines are edge triggered, no glitch must be generated on these lines. If a falling edge occurs on the external interrupt line while writing to the EXTI_FTSR register, the pending bit is not set.

Rising and falling edge triggers can be set for the same interrupt line. In this configuration, both generate a trigger condition.

EXTI Register

- **EXTI Setting:**
- Configure Interrupt mask : (**EXTI_IMR**)

Example:
PC_13(user button) EXTI
Falling Edge Trigger
No masking interrupt

```
EXTI->IMR  |= EXTI_IMR_MR13;
```

10.3.1 Interrupt mask register (EXTI_IMR)

Mask = '가리다' → disable

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									MR22	MR21	Reserved		MR18	MR17	MR16
									rw	rw			rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:0 **MRx:** Interrupt mask on line x

0: Interrupt request from line x is masked

1: Interrupt request from line x is not masked

Not masked → enable

EXTI Register

- **NVIC (IRQ) Setting**
  - (optional) Configure the priority of EXTI interrupt request. (**NVIC_SetPriority**)
  - Enable EXTI interrupt request. (**NVIC_EnableIRQ**)

```
NVIC_EnableIRQ(EXTI15_10_IRQn);          // EXTI enable
NVIC_SetPriority(EXTI15_10_IRQn, 0);     // EXTI priority as 0
```

Example:
PC_13(user button) EXTI
Falling Edge Trigger
Priority 0 (highest)
IRQ: EXTI15_10

<Vector Table for STM32F411x: EXTI>

6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000 0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000 005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000 0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000 0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000 0068
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000 00E0

EXTI Register

- **EXTI usage:**
- Create user codes in the ISR handler
- Then, must clear pending bit after interrupt call ends

EXTIxx_IRQHandler(void)
EXTI->PR |=EXTI_PR_PRx

10.3.6 Pending register (EXTI_PR)

Address offset: 0x14
Reset value: undefined

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									PR22	PR21	Reserved		PR18	PR17	PR16
									rc_w1	rc_w1			rc_w1	rc_w1	rc_w1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:0 **PRx:** Pending bit
0: No trigger request occurred
1: selected trigger request occurred
This bit is set when the selected edge event arrives on the external interrupt line.
This bit is cleared by programming it to '1'.

Example:

```
// EXTI_PR_PR13= 1<<13;
void EXTI15_10_IRQHandler(void) {
  if ((EXTI->PR & EXTI_PR_PR13) == EXTI_PR_PR13) {
    LED_Toggle();
    EXTI->PR |= EXTI_PR_PR13; // cleared by writing '1'
  }
}
```

```
void EXTI15_10_IRQHandler(void) {
  if (is_pending_EXTI(13));
      LED_toggle();
      clear_pending_EXTI(uint32_t pin);
  }
}
```

Example Code

Example:

PC_13(user button) EXTI

Falling Edge Trigger

```c
#include "stm32f411xe.h"
#include "ecGPIO.h"
#include "ecRCC.h"
#include "ecEXTI.h"

int flag = 0;
// Initialiization
void setup(void)
{
  RCC_HSI_init();
  LED_init();
  EXTI_init(GPIOC,BUTTON_PIN,FALL,0);
  GPIO_init(GPIOC, BUTTON_PIN, EC_DIN);
  GPIO_pudr(GPIOC, BUTTON_PIN, EC_PD);

}

int main(void) {
  // Initialiization --------------------------------------------------------
    setup();

  // Inifinite Loop ---------------------------------------------------------
  while(1){}
}

void EXTI15_10_IRQHandler(void) {
  if (is_pending_EXTI(BUTTON_PIN)) {
    LED_toggle();
    clear_pending_EXTI(BUTTON_PIN); // cleared by writing '1'
  }
}

```

Exercise

1. **Tutorial_EXTI**
2. **LAB_EXTI**